

รายงานปฏิบัติการกลุ่ม

การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack โดยใช้ภาษา Java

กลุ่มที่ 5

Emergency Workflow: Event Stack และ State Machine

Algorithm A: Event Stack Method | Algorithm B: Event Stack ร่วมกับ State Machine

ลำดับ	ชื่อ-นามสกุล	รหัสนักศึกษา	บทบาทในกลุ่ม
1	สิทธิโชค ดีเหลือ	6612250018	จัดทำ report / slide
2	ภัทรกร อนุกุล	68122250068	จัดทำ Code / Flowchart
3	ระพีพัฒน์ สระทองอ่อน	68122250065	จัดทำ AI usage Log / Test Case / Pseudocode
4	สุทธาทิพย์ เพ็ชรแดง	66122250085	จัดทำ slide เพิ่มเติม
5	ชอฟูรอห์ เกละสมาน	68122250112	จัดทำ reportเพิ่มเติม
6	ศุภณัฐ กาฬภักดี	67122250053	จัดทำ slide เพิ่มเติม

บทที่ 1: บทนำ

1.1 ที่มาและความสำคัญ

ศูนย์รับแจ้งเหตุฉุกเฉิน (Emergency Call Center) ต้องบันทึกลำดับการดำเนินงานของแต่ละเคสอย่างเคร่งครัด ตั้งแต่รับแจ้งเหตุจนถึงปิดเคส เพราะการดำเนินการผิดพลาด (เช่น ปิดเคสก่อนไปถึงที่เกิดเหตุ) อาจนำไปสู่ความผิดพลาด ในการบริหารจัดการสถานการณ์ฉุกเฉิน และอาจกระทบต่อความปลอดภัยของประชาชน ระบบจึงจำเป็นต้องมีกลไกป้องกันการ

เพิ่ม Action ผิดลำดับ และในขณะเดียวกันต้องรองรับการแก้ไขข้อผิดพลาดของเจ้าหน้าที่ผ่านการ Undo และ Redo ได้เช่นเดียวกับ ระบบซอฟต์แวร์ทั่วไป โครงสร้างข้อมูล Stack จึงถูกเลือกใช้เพราะคุณสมบัติ Last-In-First-Out (LIFO) สอดคล้องโดยตรง กับความหมายของ Undo (ย้อนสิ่งที่ทำล่าสุด) และ Redo (ทำสิ่งที่เพ่งย้อนกลับไปอีกครั้ง)

1.2 Case Story

ศูนย์รับแจ้งเหตุฉุกเฉินบันทึกลำดับ Action ของแต่ละเคสตามลำดับที่ถูกต้องดังนี้ CALL_RECEIVED → TEAM_ASSIGNED → VEHICLE_DISPATCHED → ARRIVED_AT_SCENE → CASE_CLOSED ระบบต้องป้องกันไม่ให้เพิ่ม Action ผิดลำดับ (เช่น สั่งปิดเคสก่อนถึงที่เกิดเหตุ) และต้องรองรับการ Undo และ Redo การกระทำล่าสุดได้ โดยเมื่อผู้ใช้เพิ่ม Action ใหม่หลังจากเคย Undo ไปบางส่วน ประวัติใน Redo Stack ต้องถูกล้างทั้งหมดทันที เพื่อรักษาความสอดคล้องของ Workflow

1.3 วัตถุประสงค์

- 1.วิเคราะห์ Input, Output และข้อจำกัดของปัญหา Emergency Workflow
- 2.ออกแบบอัลกอริทึม 2 วิธีที่แก้ปัญหเดียวกันและให้ผลลัพธ์ถูกต้องตรงกัน
- 3.เขียน Pseudocode และอธิบายขั้นตอนการทำงานของแต่ละอัลกอริทึม
- 4.พิสูจน์ความถูกต้อง (Correctness) ของอัลกอริทึมหลักด้วย Loop Invariant
- 5.วิเคราะห์ Time Complexity และ Space Complexity ของทุกฟังก์ชันในระบบ
- 6.พัฒนาโปรแกรมภาษา Java ที่ใช้งานได้จริงพร้อม Input Validation และ Error Handling
- 7.ออกแบบและทดสอบ Test Cases ครอบคลุมกรณีปกติและกรณีขอบเขต
- 8.ทดลองวัดประสิทธิภาพจริงและเปรียบเทียบกับทฤษฎีการวิเคราะห์ Big-O

1.4 ขอบเขตงาน

1. รองรับ Action 5 ประเภทตามลำดับ Workflow คงที่ (Linear Workflow ไม่มีการแตกสาขา)
2. ตรวจสอบความถูกต้องของลำดับ Action ผ่าน Transition Table ก่อนบันทึกทุกครั้ง
3. รองรับคำสั่ง Undo และ Redo พร้อมกฎการล้าง Redo Stack เมื่อมีการเพิ่ม Action ใหม่หลัง Undo บางส่วน
4. เปรียบเทียบอัลกอริทึม 2 แบบ คือ การคำนวณสถานะจาก Event Stack ทุกครั้ง (Algorithm A) กับการเก็บสถานะ ปัจจุบันเป็นตัวแปร Cache ที่อัปเดตร่วมกับ State Machine (Algorithm B)
5. ไม่รวมถึงระบบหลายเคสพร้อมกัน (Multi-case concurrency) หรือการบันทึกข้อมูลถาวรลงฐานข้อมูล

บทที่ 2: การวิเคราะห์ปัญหา

2.1 Input

1. คำสั่งจากผู้ใช้ 1 ใน 3 ประเภท ได้แก่ ADD <ActionType>, UNDO, REDO
2. ActionType ที่รับได้มีเพียง 5 ค่า: CALL_RECEIVED, TEAM_ASSIGNED, VEHICLE_DISPATCHED, ARRIVED_AT_SCENE, CASE_CLOSED
3. สถานะปัจจุบันของ Event Stack และ Redo Stack ก่อนรับคำสั่งใหม่แต่ละครั้ง

2.2 Output

1. ค่า boolean แสดงว่าคำสั่ง ADD สำเร็จ (ลำดับถูกต้อง) หรือถูกปฏิเสธ (ลำดับผิด)
2. currentState หลังทำคำสั่งทุกครั้ง (NEW, RECEIVED, ASSIGNED, DISPATCHED, ON_SCENE, CLOSED)
3. รายการ Event History (Event Stack จาก Bottom ไป Top) และ Redo History (Redo Stack จาก Top ไป Bottom)
4. จำนวน Operation สะสม (Push/Pop/Comparison) สำหรับการวัดประสิทธิภาพ

2.3 ข้อจำกัดของปัญหา (Constraints)

1. Workflow เป็นแบบเส้นตรง (Linear) มี Transition ที่ถูกต้องเพียงเส้นทางเดียวเท่านั้น ไม่มีการแตกสาขา
2. การเพิ่ม Action ใหม่ทำได้เฉพาะเมื่อสอดคล้องกับ currentState ตาม Transition Table เท่านั้น
3. หลังเคสถูกปิด (CLOSED) จะไม่มี Transition ใด ๆ ต่อไปได้อีก (CLOSED เป็น Terminal State)
4. การเพิ่ม Action ใหม่หลังจากมีการ Undo/Redo บางส่วน ต้องล้าง Redo Stack ทั้งหมดทันที

2.4 Assumptions

1. ในหนึ่งครั้งของการทำงาน ระบบดูแลเพียง 1 เคสเท่านั้น (ไม่มีหลายเคสพร้อมกัน)
2. Action ทุกตัวที่ถูก Push เข้า Event Stack ถือว่าผ่านการตรวจสอบลำดับเรียบร้อยแล้ว จึงไม่มี Action ที่ผิดลำดับหลงเหลือใน Stack
3. currentState เริ่มต้นเสมอที่ NEW เมื่อเริ่มเคสใหม่หรือหลัง Reset
4. ระบบทำงานแบบ Single-thread จึงไม่ต้องพิจารณาปัญหา Concurrency

2.5 กรณีปกติ กรณีขอบเขต และกรณีที่อาจทำให้โปรแกรมผิดพลาด

กรณีปกติ (Normal Case)

ทำ Workflow ครบทั้ง 5 ขั้นตอนตามลำดับที่ถูกต้องตั้งแต่ CALL_RECEIVED จนถึง CASE_CLOSED

กรณีขอบเขต (Boundary / Edge Case)

1. Undo เมื่อ Event Stack ว่าง (ไม่มี Action ให้ย้อนกลับ)
2. Redo เมื่อ Redo Stack ว่าง (ไม่มี Action ให้ทำซ้ำ)
3. เพิ่ม Action หลังจากเคสถูกปิดแล้ว (CLOSED เป็น Terminal State)
4. Undo จนกลับสู่สถานะ NEW ตั้งแต่เริ่มต้น

กรณีที่ทำให้โปรแกรมผิดพลาด (Error-Prone Case)

1. เพิ่ม Action ที่ข้ามขั้นตอน เช่น สั่ง ARRIVED_AT_SCENE ทันทีหลัง CALL_RECEIVED โดยไม่ผ่าน TEAM_ASSIGNED และ VEHICLE_DISPATCHED
2. สั่งปิดเคส (CASE_CLOSED) ก่อนถึงที่เกิดเหตุ (ก่อน ARRIVED_AT_SCENE)
3. เพิ่ม Action ใหม่หลัง Redo บางส่วนโดยไม่ล้าง Redo Stack ให้ถูกต้อง จะทำให้ currentState กับ Redo Stack ไม่สอดคล้องกัน
4. รับ Input ที่ไม่ตรงกับชื่อ ActionType ทั้ง 5 ค่า (Invalid Input)

2.6 Operation ที่เกิดขึ้นบ่อยและ Operation ที่อาจใช้เวลามากที่สุด

Operation ที่เกิดขึ้นบ่อยที่สุดในระบบคือ addAction() เนื่องจากเป็นคำสั่งหลักที่เจ้าหน้าที่ใช้บันทึกความคืบหน้าของเคสตลอดกระบวนการ รองลงมาคือการแสดงสถานะปัจจุบัน (getCurrentState / displayStatus)

Operation ที่อาจใช้เวลามากที่สุดคือ deriveCurrentState() ใน Algorithm A ซึ่งต้องไล่ Pop ทุก Action ออกจาก Event Stack ไปเก็บใน Temporary Stack แล้ว Pop กลับเพื่อคำนวณสถานะใหม่ที่ละชั้น ก่อน Push คืนกลับ Event Stack เดิม ทำให้มี Time Complexity $O(n)$ และถูกเรียกใช้ทุกครั้งที่ addAction() หรือ getCurrentState() ถูกเรียก ต่างจาก Algorithm B ที่เก็บ currentState เป็นตัวแปร Cache จึงไม่ต้องสแกน Stack เลย

บทที่ 3: การออกแบบอัลกอริทึม

3.1 Algorithm A: Event Stack Method

แนวคิดหลัก: ไม่เก็บ currentState เป็นตัวแปรแยกต่างหาก ทุกครั้งที่ต้องตรวจสอบ Action ใหม่หรืออ่านสถานะปัจจุบัน อัลกอริทึมจะ "ค้นหา" (สแกน) Event Stack ทั้งหมดตั้งแต่ตัวแรก (Bottom) ผ่าน Transition Table ทีละชั้น เพื่อคำนวณสถานะปัจจุบันใหม่ทุกครั้ง โดยใช้ Temporary Stack ช่วยไล่ลำดับแล้ว Restore กลับ

Input / Output

1. Input: eventStack, redoStack (Deque<EventAction>), action ที่ต้องการเพิ่ม (สำหรับ addAction) หรือไม่ต้องการ argument เพิ่มเติม (สำหรับ undo/redo)
2. Output: boolean แสดงผลสำเร็จ/ล้มเหลว และ currentState ที่คำนวณใหม่ (WorkflowState)

Pseudocode

```

ALGORITHM DeriveCurrentState_A(eventStack)
INPUT:  eventStack คือ Stack ของ Action ทั้งหมดที่บันทึกไว้
        (n รายการ)
OUTPUT: currentState ที่คำนวณได้จากลำดับ Action ใน
        eventStack
1.  Create an empty temporaryStack
2.  WHILE eventStack is not empty
3.    temporaryStack.push(eventStack.pop())
4.  END WHILE
5.  state <- NEW
6.  WHILE temporaryStack is not empty
7.    action <- temporaryStack.pop()
8.    state <- TransitionTable.getNextState(state,
        action)
9.    eventStack.push(action)      // Restore กลับสู่
        eventStack เดิม
10. END WHILE
11. RETURN state

ALGORITHM AddAction_A(eventStack, redoStack, action)
INPUT:  eventStack, redoStack, action ที่ต้องการเพิ่ม
OUTPUT: true หากลำดับถูกต้องและเพิ่มสำเร็จ, false หากผิดลำดับ
1.  currentState <- DeriveCurrentState_A(eventStack)
    // O(n)

```

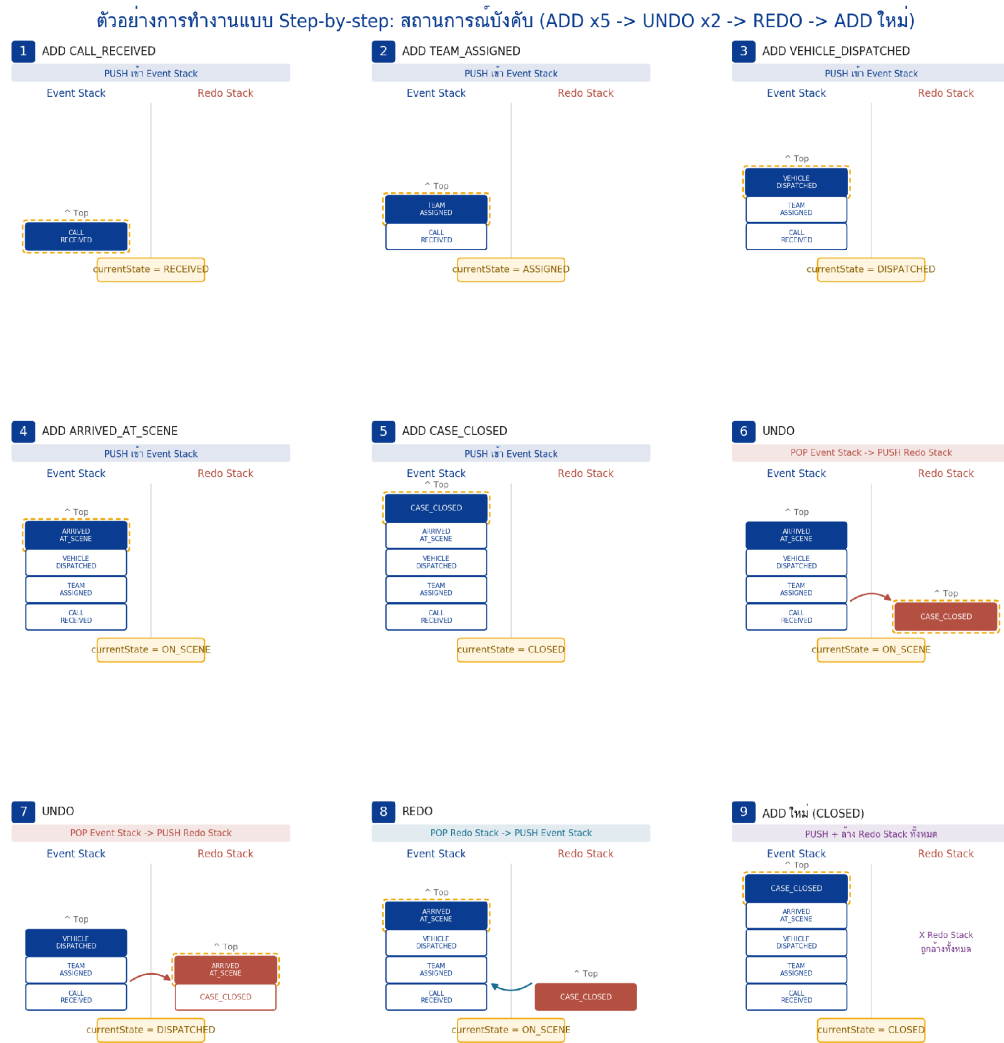
```

2. nextState <-
TransitionTable.getNextState(currentState, action)
3. IF nextState = NULL THEN
4.   RETURN false
5. END IF
6. eventStack.push(action)
7. IF redoStack is not empty THEN Clear redoStack
8. RETURN true

```

ตัวอย่างการทำงาน (Step-by-step)

พิจารณาลำดับคำสั่ง ADD CALL_RECEIVED, ADD TEAM_ASSIGNED, ..., ADD CASE_CLOSED แล้วตามด้วย UNDO, UNDO, REDO และ ADD Action ใหม่ (สถานการณ์บังคับของโจทย์) ผลลัพธ์ในแต่ละขั้นตอนของ Event Stack และ Redo Stack แสดงในภาพที่ 3.1 ด้านล่าง (ผลลัพธ์เหมือนกันทั้ง Algorithm A และ B เพราะทั้งสองวิธีต้องให้ผลลัพธ์ถูกต้องตรงกันตามข้อกำหนด)



ภาพที่ 3.1 สถานะของ Event Stack และ Redo Stack ในแต่ละขั้นตอนของสถานการณ์บังคับ

ข้อดีและข้อจำกัด

- ข้อดี: โครงสร้างข้อมูลเรียบง่าย มี Single Source of Truth เดียวคือ Event Stack ไม่มีการเก็บ State ซ้ำซ้อนที่อาจไม่ตรงกัน (Consistency รับประกันโดยธรรมชาติ)
- ข้อจำกัด: การตรวจสอบ Action ใหม่ทุกครั้งต้อง Pop/Push ทั้ง Stack เพื่อคำนวณสถานะใหม่ ทำให้ addAction() และ getCurrentState() มี Time Complexity $O(n)$

Time Complexity และ Space Complexity (สรุปย่อ)

- addAction / getCurrentState: $O(n)$ — ต้องสแกน Event Stack ทั้งหมดทุกครั้ง
- undo / redo: $O(1)$ — ย้าย Action เดียวระหว่าง eventStack กับ redoStack โดยไม่ต้องสแกน
- Auxiliary Space: $O(n)$ — ใช้ Temporary Stack ขนาดเท่ากับ eventStack ระหว่างการคำนวณสถานะ

3.2 Algorithm B: Event Stack ร่วมกับ State Machine

แนวคิดหลัก: เก็บ `currentState` เป็นตัวแปรเดียว (Cache ของสถานะล่าสุด) ทุกครั้งที่เพิ่ม Action ใหม่หรือ Undo/Redo ใช้ Transition Table ตรวจสอบและอัปเดต `currentState` ทันทีแบบ Incremental โดยไม่ต้องไล่สแกน Event Stack ทั้งหมด

Input / Output

1. Input: `eventStack`, `redoStack`, `currentState` (ตัวแปร Cache), action ที่ต้องการเพิ่ม
2. Output: boolean แสดงผลสำเร็จ/ล้มเหลว และ `currentState` ที่อัปเดตแล้ว

Pseudocode

```

ALGORITHM AddAction_B(eventStack, redoStack, currentState, action)

INPUT: eventStack, redoStack, currentState ปัจจุบัน, action ที่ต้องการเพิ่ม
OUTPUT: true หากลำดับถูกต้องและเพิ่มสำเร็จ, false หากผิดลำดับ
        (currentState ถูกอัปเดตโดยอ้างอิง หาก Transition สำเร็จ)

1. nextState <- TransitionTable.getNextState(currentState, action) // O(1)
2. IF nextState = NULL THEN
3.     RETURN false
4. END IF
5. eventStack.push(action)
6. currentState <- nextState
7. IF redoStack is not empty THEN Clear redoStack
8. RETURN true

ALGORITHM Undo_B(eventStack, redoStack, currentState)

1. IF eventStack is empty THEN RETURN false
2. lastAction <- eventStack.pop()
3. redoStack.push(lastAction)
4. currentState <- TransitionTable.getPreviousState(currentState, lastAction)
  
```

```
5. RETURN true
```

```
ALGORITHM Redo_B(eventStack, redoStack, currentState)
```

- ```
1. IF redoStack is empty THEN RETURN false
2. action <- redoStack.pop()
3. nextState <- TransitionTable.getNextState(currentState, action)
4. eventStack.push(action)
5. currentState <- nextState
6. RETURN true
```

### ตัวอย่างการทำงาน (Step-by-step)

ใช้สถานการณ์บังคับเดียวกันกับ Algorithm A (ภาพที่ 3.1) — Algorithm B ให้ผลลัพธ์ Event Stack, Redo Stack และ currentState ตรงกันทุกขั้นตอน เพียงแต่ currentState ถูกอ่านจากตัวแปร Cache แทนการสแกน Stack

### ข้อดีและข้อจำกัด

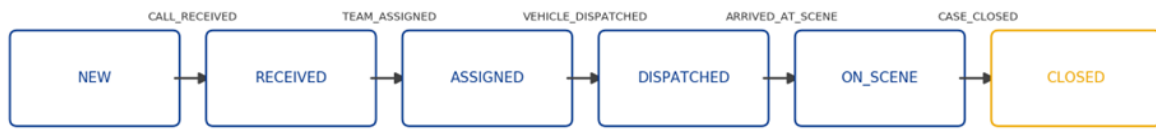
- ข้อดี: ตรวจสอบ Action ใหม่และอ่านสถานะปัจจุบันได้ใน  $O(1)$  เหมาะกับระบบที่ต้องตรวจสอบสถานะบ่อยและมี Event จำนวนมาก
- ข้อจำกัด: ต้องดูแลให้ currentState สอดคล้องกับ Event Stack เสมอ (Invariant) หากมีจุดใดอัปเดต currentState ผิดพลาด (เช่น ลืมอัปเดตในบางเส้นทางของโค้ด) ระบบจะเสีย Consistency ทันทีโดยไม่มีกลไกตรวจจับอัตโนมัติ

### Time Complexity และ Space Complexity (สรุปย่อ)

- addAction / getCurrentState:  $O(1)$  — ใช้ Transition Table แบบ HashMap และตัวแปร Cache
- undo / redo:  $O(1)$  — getPreviousState ไต่ตาราง Transition ที่มีขนาดคงที่ (6 สถานะ) จึงถือเป็นค่าคงที่
- Auxiliary Space:  $O(1)$  เพิ่มเติมจาก Event Stack เดิม (ไม่ต้องสร้าง Temporary Stack)

### 3.3 ความสัมพันธ์กับ State Machine (Transition Table)

ทั้งสองอัลกอริทึมใช้ Transition Table เดียวกันเป็นแหล่งอ้างอิงกฎการเปลี่ยนสถานะ ดังภาพที่ 3.2 ซึ่ง Implement ด้วย Nested HashMap<WorkflowState, HashMap<ActionType, WorkflowState>> ทำให้การค้นหา Next State ทำได้ใน  $O(1)$  เสมอ ไม่ว่าจะเรียกจาก Algorithm A หรือ B



Transition Table: currentState + action -> nextState (O(1) lookup, HashMap<WorkflowState, HashMap<ActionType, WorkflowState>>)

ภาพที่ 3.2 แผนภาพ State Transition ของ Emergency Workflow (NEW → RECEIVED → ASSIGNED → DISPATCHED → ON\_SCENE → CLOSED)

#### บทที่ 4: การอธิบายความถูกต้อง (Correctness)

กลุ่มเลือก Algorithm B (Event Stack ร่วมกับ State Machine) เป็นอัลกอริทึมหลักในการพิสูจน์ความถูกต้อง เนื่องจากมีความซับซ้อนของ Invariant สูงกว่า (ต้องรักษาความสอดคล้องระหว่างตัวแปร Cache กับโครงสร้างข้อมูล Stack) และสอดคล้องกับ Correctness Invariant ที่โจทย์กำหนดไว้ โดยใช้แนวทาง Loop Invariant ที่พิจารณาลำดับการเรียกใช้ Operation (ADD / UNDO / REDO) ต่อเนื่องกันเสมือนเป็นรอบของ Loop

##### 4.1 นิยาม Invariant

Invariant: ก่อนและหลังจากทุก Operation (ADD, UNDO, REDO) ค่า currentState ต้องเท่ากับผลลัพธ์ของการไล่ Transition Table ตามลำดับ Action ทั้งหมดใน Event Stack จาก Bottom ไป Top เริ่มจาก NEW และลำดับ Action ใน Event Stack ต้องเป็นลำดับ Workflow ที่ถูกต้องเสมอ (กล่าวคือไม่มี Action ใดผิดลำดับหลงเหลืออยู่ใน Stack)

##### 4.2 Initialization (ก่อนเริ่มอัลกอริทึม)

ก่อนเริ่มทำงาน Event Stack และ Redo Stack ว่างเปล่า และ currentState ถูกกำหนดเป็น NEW โดยตรง (ในบรรทัด private WorkflowState currentState = WorkflowState.NEW;) เนื่องจากไม่มี Action ใดใน Event Stack เลย การไล่ Transition Table จากลำดับ Action ว่างจึงให้ผลเป็น NEW เช่นกัน ดังนั้น Invariant เป็นจริงตั้งแต่อันแรก (Base Case)

##### 4.3 Maintenance (ระหว่างทำงาน คุณสมบัติโดยยังคงเป็นจริง)

กรณี addAction(action)

ฟังก์ชันจะคำนวณ nextState จาก TransitionTable.getNextState(currentState, action) ก่อนเสมอ หาก nextState เป็น null (ไม่มี Transition ที่ถูกต้อง) ฟังก์ชันจะ RETURN false ทันทีโดยไม่ Push Action และไม่แก้ไข

currentState — Event Stack และ currentState จึงไม่เปลี่ยนแปลง Invariant ยังคงเป็นจริงโดยอัตโนมัติ หาก nextState ถูกต้อง ฟังก์ชันจะ Push action เข้า Event Stack และกำหนด currentState <- nextState พร้อมกันในระดับอะตอมเดียว (Atomic ในเชิงลำดับการทำงาน) ทำให้ Action ล่าสุดใน Stack กับ currentState ยังคงสอดคล้องกันตาม Transition Table เสมอ

#### กรณี undo()

หาก Event Stack ว่าง ฟังก์ชันจะ RETURN false ทันทีโดยไม่แก้ไขค่าใด ๆ Invariant คงอยู่ หาก Event Stack ไม่ว่าง ฟังก์ชันจะ Pop lastAction ออกจาก Event Stack และย้ายไปยัง Redo Stack จากนั้นคำนวณ currentState ใหม่ด้วย TransitionTable.getPreviousState(currentState, lastAction) ซึ่งเป็นการย้อนกลับ Transition เดียวกับที่ใช้ตอน Push lastAction เข้ามา จึงรับประกันว่า currentState หลัง Undo จะตรงกับผลของการไล่ Transition Table จาก Action ที่เหลือใน Event Stack เสมอ

#### กรณี redo()

หาก Redo Stack ว่าง ฟังก์ชันจะ RETURN false ทันทีโดยไม่แก้ไขค่าใด ๆ หาก Redo Stack ไม่ว่าง ฟังก์ชันจะ Pop action ล่าสุดจาก Redo Stack, คำนวณ nextState จาก currentState ปัจจุบัน (ซึ่งถูกต้องตาม Invariant อยู่แล้วจาก Operation ก่อนหน้า) แล้ว Push action กลับเข้า Event Stack พร้อมอัปเดต currentState <- nextState ในขั้นตอนเดียวกัน Invariant จึงคงอยู่เช่นเดียวกับกรณี addAction

#### กฎการล้าง Redo Stack

เมื่อ addAction() สำเร็จและ Redo Stack ไม่ว่าง ระบบจะ Clear Redo Stack ทันที ป้องกันไม่ให้ Action เก่าที่เคยถูก Undo ไปถูกนำกลับมาใช้ Redo ผิดบริบท (เช่น Redo Action ที่ไม่สอดคล้องกับ currentState ใหม่) จุดนี้เป็นส่วนสำคัญที่ทำให้ Invariant "ลำดับ Action ใน Event Stack เป็น Workflow ที่ถูกต้องเสมอ" ไม่ถูกละเมิดหลังจากผู้ใช้ทำ Action ใหม่ที่ต่างจากเส้นทางเดิม

#### 4.4 Termination (เมื่ออัลกอริทึมหยุด ผลลัพธ์ถูกต้องหรือไม่)

ทุก Operation (addAction, undo, redo) เป็นการทำงานแบบจำกัดขั้นตอน (ไม่มี Loop ที่ขึ้นกับ n ใน Algorithm B) จึงหยุดทำงานได้เสมอในเวลาจำกัด (Termination รับประกันโดยโครงสร้าง). เมื่อผู้ใช้หยุดป้อนคำสั่ง สถานะสุดท้ายของระบบจะสอดคล้องกับ Invariant เสมอ นั่นคือ currentState เท่ากับผลลัพธ์ของการไล่ Transition Table จาก Action ทั้งหมดใน Event Stack และลำดับ Action ใน Event Stack เป็น Workflow ที่ถูกต้อง — สอดคล้องกับผลการทดสอบจริงในบทที่ 7 ที่ TestCases ทั้ง 10 กรณีผ่านครบสำหรับทั้งสองอัลกอริทึม (20/20 PASS)

#### 4.5 ข้อมูลที่ไม่เกี่ยวข้องสูญหายหรือเปลี่ยนลำดับหรือไม่

Action ที่ถูก Undo จะถูกย้ายไปเก็บใน Redo Stack แบบครบถ้วนโดยไม่มีการลบทิ้ง (จนกว่าจะถูกล้างด้วยกฎ Redo Stack) และ Action ที่เหลือใน Event Stack จะไม่ถูกสลับลำดับ เนื่องจากทุก Operation ทำงานที่ปลาย Stack (Top) เพียงตำแหน่งเดียวเท่านั้น (Push/Pop) จึงไม่มีการเปลี่ยนลำดับของ Action ที่ไม่เกี่ยวข้องกับ Operation ปัจจุบัน

#### 4.6 สรุปการพิสูจน์ความถูกต้อง

จาก Initialization ที่ Invariant เป็นจริงตั้งแต่ต้น, Maintenance ที่แสดงว่าทุก Operation รักษา Invariant ไว้ได้ ไม่ว่าผลลัพธ์จะสำเร็จหรือถูกปฏิเสธ, และ Termination ที่แสดงว่าระบบหยุดทำงานได้เสมอพร้อมผลลัพธ์ที่ยังสอดคล้องกับ Invariant จึงสรุปได้ว่า Algorithm B ทำงานถูกต้องตามข้อกำหนดของโจทย์ในทุกกรณี และเนื่องจาก Algorithm A ถูกออกแบบให้ addAction คำนวณ currentState ใหม่จาก Event Stack ทั้งหมดทุกครั้งก่อนตัดสินใจ (ไม่ได้พึ่งพา Cache ที่อาจผิดพลาด) จึงรักษา Invariant เดียวกันได้โดยอัตโนมัติเช่นกัน (Correctness by Construction) ผลการทดสอบจริงยืนยันว่าทั้งสองอัลกอริทึมให้ผลลัพธ์ตรงกันทุก Test Case

#### บทที่ 5: การวิเคราะห์ประสิทธิภาพ

ตารางด้านล่างวิเคราะห์ทุกฟังก์ชันหลักของระบบ (ไม่ใช่เพียง push/pop) โดย n คือจำนวน Action ทั้งหมดที่อยู่ใน Event Stack ณ ขณะนั้น

##### 5.1 ตารางเปรียบเทียบ Time Complexity รายฟังก์ชัน

(A = Algorithm A: Event Stack Method, B = Algorithm B: Event Stack + State Machine)

| ฟังก์ชัน                               | A-Best | A-Avg  | A-Worst | B-Best | B-Avg  | B-Worst |
|----------------------------------------|--------|--------|---------|--------|--------|---------|
| addAction()                            | $O(n)$ | $O(n)$ | $O(n)$  | $O(1)$ | $O(1)$ | $O(1)$  |
| undo()                                 | $O(1)$ | $O(1)$ | $O(1)$  | $O(1)$ | $O(1)$ | $O(1)$  |
| redo()                                 | $O(1)$ | $O(1)$ | $O(1)$  | $O(1)$ | $O(1)$ | $O(1)$  |
| getCurrentState()                      | $O(n)$ | $O(n)$ | $O(n)$  | $O(1)$ | $O(1)$ | $O(1)$  |
| getEventHistory() /<br>displayStatus() | $O(n)$ | $O(n)$ | $O(n)$  | $O(n)$ | $O(n)$ | $O(n)$  |
| deriveCurrentState() [เฉพาะ<br>A]      | $O(n)$ | $O(n)$ | $O(n)$  | -      | -      | -       |

หมายเหตุ: สำหรับ `addAction` ของ Algorithm A ไม่มี Best Case ที่ดีกว่า  $O(n)$  เพราะฟังก์ชันต้องเรียก `deriveCurrentState()` เพื่อคำนวณ `currentState` ก่อนตัดสินใจทุกครั้ง ไม่ว่า Action ที่ส่งเข้ามาจะถูกหรือผิดพลาดก็ตาม ในขณะที่ Algorithm B ตรวจสอบผ่านตัวแปร `Cache` จึงเป็น  $O(1)$  ในทุกกรณี ส่วน `undo()`/`redo()` ของทั้งสองอัลกอริทึมเป็น  $O(1)$  เพราะทำงานที่ปลาย Stack เพียงตำแหน่งเดียว และ `getPreviousState()` ไล่ตาราง Transition ที่มีขนาดคงที่ (6 แถว) จึงถือเป็นค่าคงที่ ( $O(1)$ )

## 5.2 Space Complexity

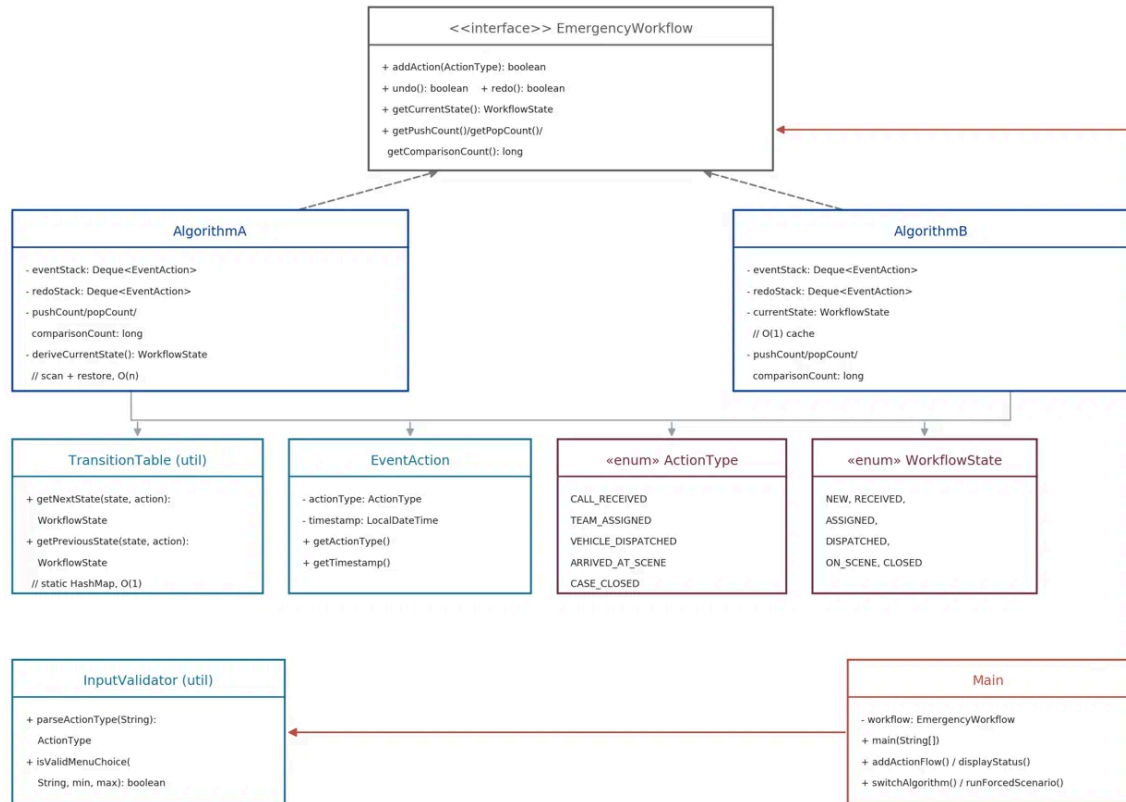
| รายการ                                                                   | Algorithm A                                                   | Algorithm B                               |
|--------------------------------------------------------------------------|---------------------------------------------------------------|-------------------------------------------|
| โครงสร้างข้อมูลหลัก ( <code>eventStack</code> + <code>redoStack</code> ) | $O(n)$                                                        | $O(n)$                                    |
| Auxiliary Space ระหว่างทำงาน                                             | $O(n)$ — Temporary Stack ใน <code>deriveCurrentState()</code> | $O(1)$ — ไม่ต้องใช้ Stack ชั่วคราว        |
| ตัวแปรสถานะเพิ่มเติม                                                     | ไม่มี (คำนวณสด)                                               | $O(1)$ — ตัวแปร <code>currentState</code> |
| Total Space Complexity                                                   | $O(n)$                                                        | $O(n)$                                    |

## 5.3 บทสรุปเชิงทฤษฎี

1. Algorithm A เหมาะกับระบบที่ไม่ต้องการอ่านสถานะบ่อยและต้องการลดความเสี่ยงของ State ที่ไม่ตรงกับข้อมูลจริง เพราะคำนวณจากข้อมูลตั้งต้นเสมอ
2. Algorithm B เหมาะกับระบบที่ต้องอ่าน/ตรวจสอบสถานะบ่อยและมีจำนวน Action สะสมมาก เพราะให้ประสิทธิภาพ  $O(1)$  คงที่ไม่ขึ้นกับ  $n$
3. เมื่อ  $n$  มีขนาดใหญ่ ผลต่างของ Time Complexity ระหว่าง  $O(n)$  และ  $O(1)$  จะยิ่งเห็นชัดเจนขึ้น ซึ่งสอดคล้องกับผลการทดลองจริงในบทที่ 7

## บทที่ 6: การพัฒนาโปรแกรม Java

### 6.1 Class Diagram



ภาพที่ 6.1 Class Diagram ของระบบ Emergency Workflow

## 6.2 โครงสร้างคลาส

| คลาส / Package                           | หน้าที่หลัก                                                                                                                      |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| models.ActionType (enum)                 | กำหนดประเภท Action ทั้ง 5 ค่าตามลำดับ Workflow ที่ถูกต้อง                                                                        |
| models.WorkflowState (enum)              | กำหนดสถานะของเคสทั้ง 6 สถานะ (NEW ถึง CLOSED)                                                                                    |
| models.EventAction                       | Data Model เก็บ Action พร้อม Timestamp ที่เกิดขึ้นจริง                                                                           |
| algorithms.EmergencyWorkflow (interface) | สัญญากลางที่ Algorithm A และ B ต้อง Implement ร่วมกัน เพื่อให้ Main และ PerformanceTest เรียกใช้งานแบบเดียวกันได้ (Polymorphism) |
| algorithms.AlgorithmA                    | คำนวณ currentState จากการสแกน Event Stack ทั้งหมดทุกครั้ง (Event Stack Only)                                                     |

|                             |                                                                                                 |
|-----------------------------|-------------------------------------------------------------------------------------------------|
| algorithms.AlgorithmB       | เก็บ currentState เป็นตัวแปร Cache ที่อัปเดตร่วมกับ State Machine (Event Stack + State Machine) |
| utils.TransitionTable       | ตารางกฎการเปลี่ยนสถานะแบบ Static (Nested HashMap) ค้นหา Next/Previous State ได้ใน $O(1)$        |
| utils.InputValidator        | ตรวจสอบและแปลง Input จากผู้ใช้ให้เป็น ActionType หรือหมายเลขเมนูที่ถูกต้อง                      |
| Main                        | เมนูโต้ตอบผู้ใช้ รองรับ ADD / UNDO / REDO / แสดงสถานะ / สลับอัลกอริทึม / รันสถานการณ์บังคับ     |
| experiments.PerformanceTest | ทดลองวัดประสิทธิภาพทั้งสองอัลกอริทึมตามขนาดข้อมูลที่กำหนด บันทึกผลลง CSV                        |
| TestCases                   | ชุดทดสอบ 10 กรณี รันกับทั้งสองอัลกอริทึมโดยอัตโนมัติ (รวม 20 การทดสอบ)                          |

### 6.3 เมธอดสำคัญ

| เมธอด                                      | อยู่ในคลาส      | หน้าที่                                                        |
|--------------------------------------------|-----------------|----------------------------------------------------------------|
| addAction(ActionType)                      | AlgorithmA / B  | ตรวจสอบลำดับผ่าน Transition Table แล้ว Push Action หากถูกต้อง  |
| undo() / redo()                            | AlgorithmA / B  | ย้าย Action ระหว่าง eventStack และ redoStack พร้อมปรับสถานะ    |
| deriveCurrentState()                       | AlgorithmA      | สแกน Event Stack ทั้งหมดเพื่อคำนวณ currentState ใหม่ (private) |
| getNextState() /<br>getPreviousState()     | TransitionTable | ค้นหาสถานะถัดไป/ก่อนหน้าจากตาราง Transition แบบ $O(1)$         |
| parseActionType() /<br>isValidMenuChoice() | InputValidator  | ตรวจสอบความถูกต้องของข้อความ/ตัวเลขที่ผู้ใช้ป้อน               |

## 6.4 Input Validation

ผู้ใช้ป้อน Action เป็นข้อความอิสระผ่านเมนู ระบบใช้ `InputValidator.parseActionType()` ตัดช่องว่างและแปลงเป็นตัวพิมพ์ใหญ่ก่อนเทียบกับชื่อค่าคงที่ใน `ActionType` enum หากไม่ตรงกับค่าใดเลยจะคืนค่า `null` และ Main จะแสดงข้อความ "Action ไม่ถูกต้อง (Invalid Input)" โดยไม่กระทบสถานะระบบ ส่วนตัวเลือกเมนูใช้ `InputValidator.isValidMenuChoice()` ตรวจสอบว่าค่าที่ป้อนเป็นตัวเลขและอยู่ในช่วงที่กำหนด

## 6.5 Error Handling

1. Action ผิดลำดับ: `addAction()` คืนค่า `false` โดยไม่แก้ไข Event Stack หรือ `currentState` ระบบแสดง `currentState` ปัจจุบันเพื่อแจ้งเหตุผล
2. Undo เมื่อ Event Stack ว่าง: `undo()` คืนค่า `false` ทันที ป้องกัน `NoSuchElementException` จาก `Deque.pop()` บน Stack ว่าง
3. Redo เมื่อ Redo Stack ว่าง: `redo()` คืนค่า `false` ทันทีด้วยหลักการเดียวกัน
4. Input ไม่ถูกต้อง: ตรวจสอบผ่าน `InputValidator` ก่อนเรียกใช้ Business Logic เสมอ ป้องกันข้อมูลขยะเข้าสู่ระบบ

## บทที่ 7: การทดลอง

### 7.1 การออกแบบการทดลอง

1. ขนาดข้อมูล (n จำนวน Action ทั้งหมด): 100, 1,000, 10,000 และ 50,000
2. จำนวนรอบทดลองต่อขนาดข้อมูล: 5 รอบ แล้วหาค่าเฉลี่ย (Average) ของเวลาและจำนวน Operation
3. วิธีสร้างข้อมูล: วนคำสั่ง `addAction()` ตามลำดับ Workflow ที่ถูกต้อง 5 ขั้นตอนต่อรอบ (`CALL_RECEIVED` → ... → `CASE_CLOSED`) แล้ว `reset()` เพื่อเริ่ม Workflow ใหม่ ทำซ้ำจนครบ n Action ตามที่กำหนด
4. เครื่องมือวัดเวลา: `System.nanoTime()` ก่อนและหลังการทำงานของแต่ละรอบ
5. รันจริงบน OpenJDK 21 ผ่าน `experiments.PerformanceTest.java` ผลลัพธ์บันทึกลง `results/experiment_results.csv`

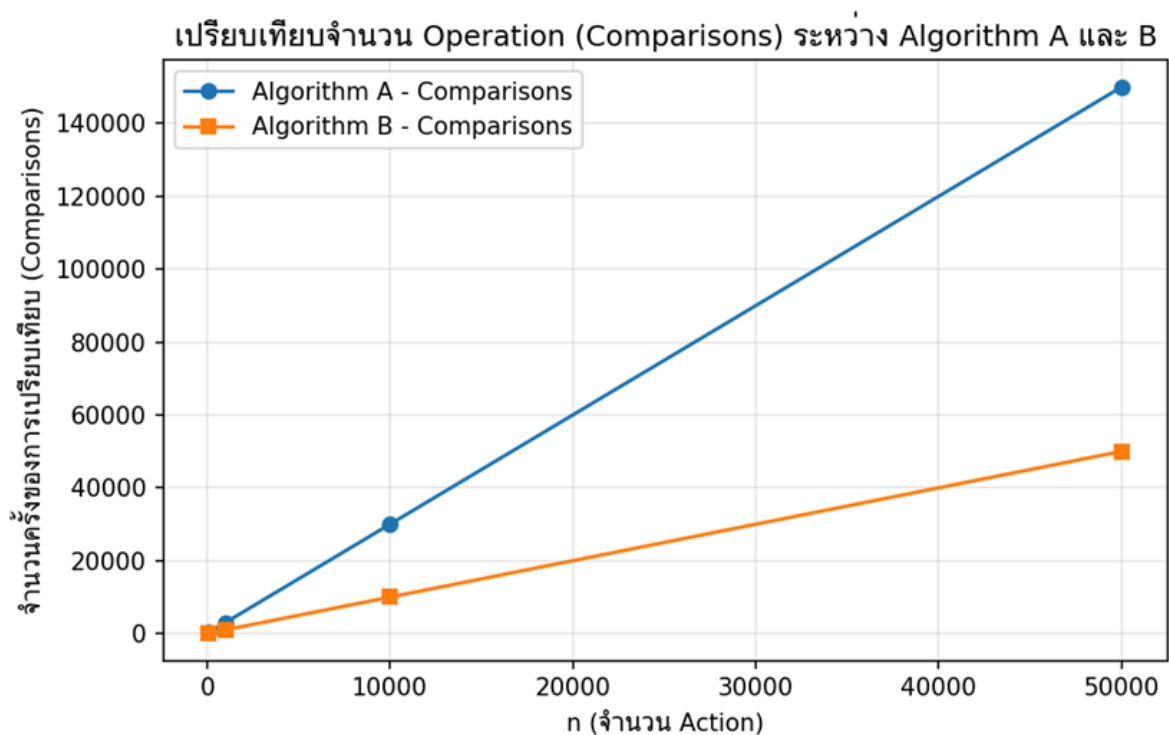
### 7.2 ตารางผลการทดลอง (ข้อมูลจริงจากการรันโปรแกรม)

| n   | Algorithm            | Avg Time<br>(ms) | Push | Pop | Comparisons |
|-----|----------------------|------------------|------|-----|-------------|
| 100 | A (Event Stack)      | 16.09            | 300  | 400 | 300         |
| 100 | B (Event Stack + SM) | 1.70             | 100  | 0   | 100         |

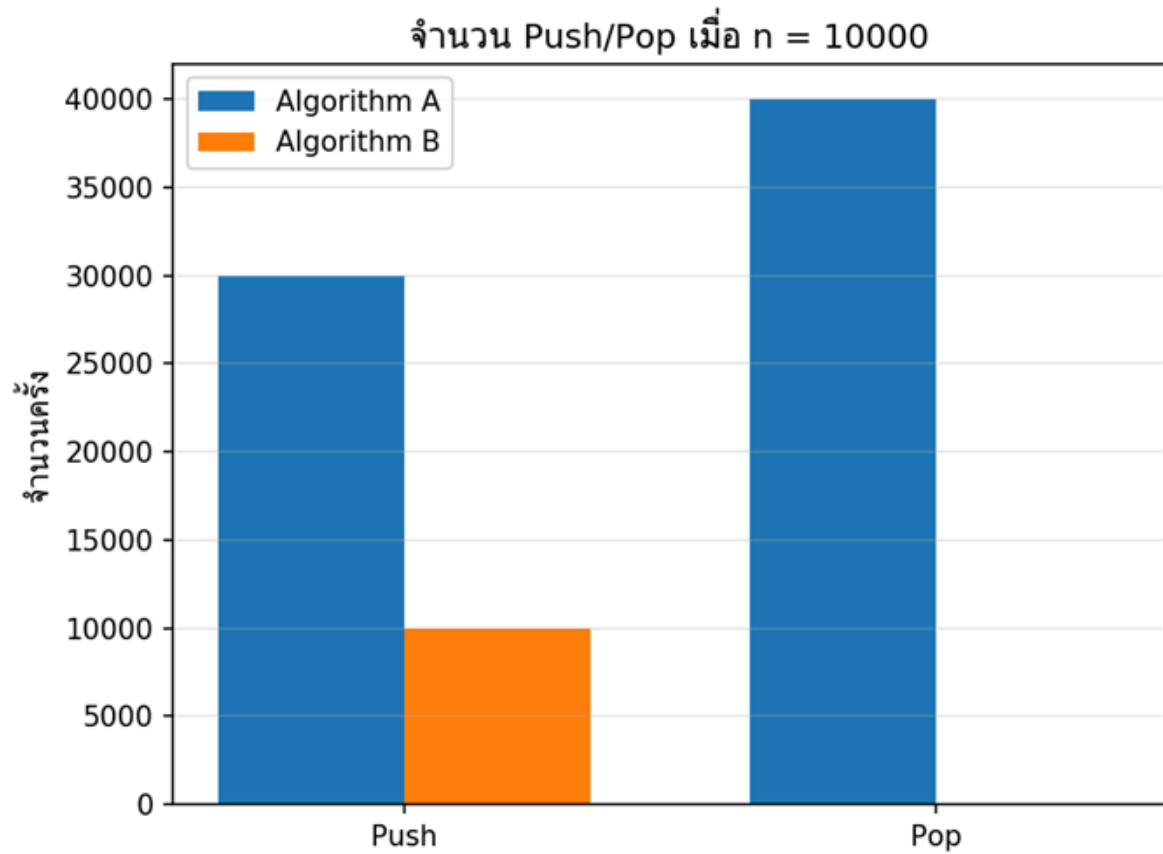
|        |                      |       |         |         |         |
|--------|----------------------|-------|---------|---------|---------|
| 1,000  | A (Event Stack)      | 3.56  | 3,000   | 4,000   | 3,000   |
| 1,000  | B (Event Stack + SM) | 1.15  | 1,000   | 0       | 1,000   |
| 10,000 | A (Event Stack)      | 21.79 | 30,000  | 40,000  | 30,000  |
| 10,000 | B (Event Stack + SM) | 5.62  | 10,000  | 0       | 10,000  |
| 50,000 | A (Event Stack)      | 20.80 | 150,000 | 200,000 | 150,000 |
| 50,000 | B (Event Stack + SM) | 28.08 | 50,000  | 0       | 50,000  |

(SM = State Machine; ค่า Avg Time เป็นหน่วยมิลลิวินาที ปัดเศษ 2 ตำแหน่ง; ข้อมูลดิบหน่วยนาโนวินาทีอยู่ในไฟล์ `results/experiment_results.csv`)

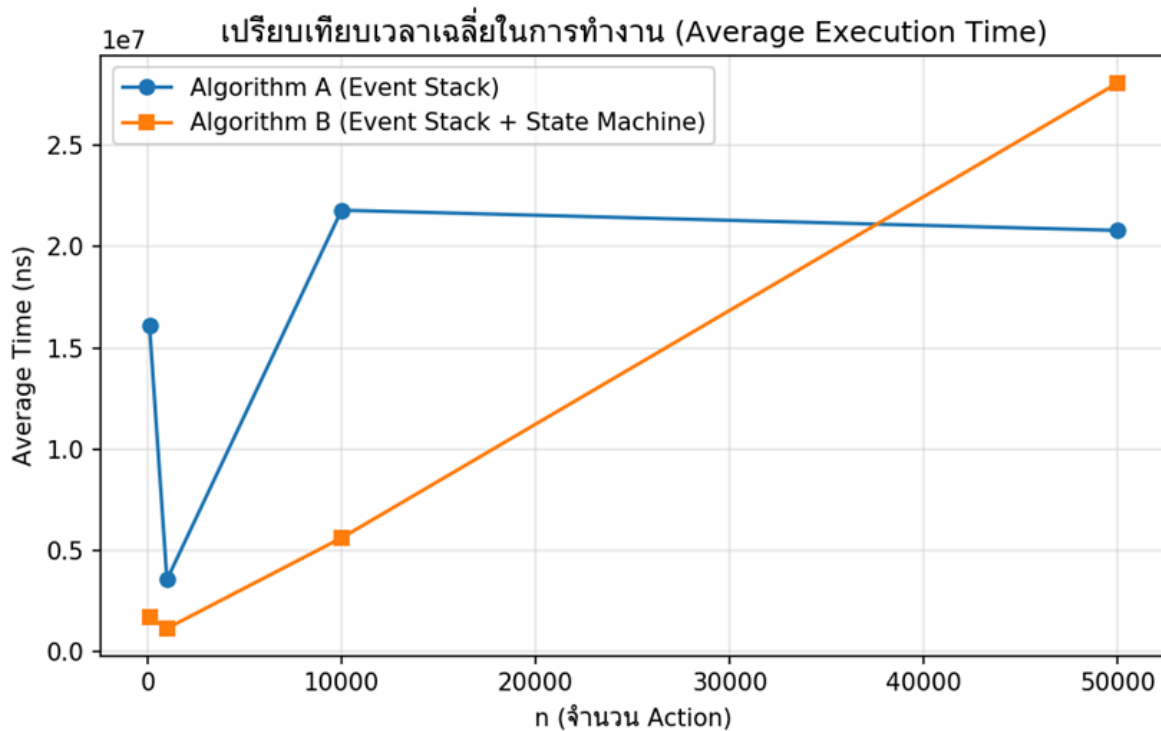
### 7.3 กราฟเปรียบเทียบ



ภาพที่ 7.1 จำนวนครั้งของการเปรียบเทียบ (Comparisons) เทียบกับขนาดข้อมูล  $n$  — Algorithm A เพิ่มแบบเชิงเส้นในอัตราที่ชันกว่า Algorithm B อย่างชัดเจน



ภาพที่ 7.2 จำนวน Push/Pop เมื่อ  $n = 10,000$  — Algorithm A มี Push และ Pop มากกว่า Algorithm B ประมาณ 3-4 เท่า เนื่องจากการ Restore Stack ใน `deriveCurrentState()`



ภาพที่ 7.3 เวลาเฉลี่ยในการทำงาน (Average Execution Time) เทียบกับขนาดข้อมูล  $n$

#### 7.4 การอภิปรายผล

จำนวน Operation (Push, Pop, Comparisons) สอดคล้องกับผลการวิเคราะห์ Big-O ในบทที่ 5 อย่างชัดเจน: Algorithm A มีจำนวน Push และ Pop สูงกว่า Algorithm B อย่างมีนัยสำคัญในทุกขนาดข้อมูล (เช่นที่  $n = 10,000$  มี Push 30,000 ครั้งเทียบกับ 10,000 ครั้ง และ Pop 40,000 ครั้งเทียบกับ 0 ครั้ง) เพราะ `deriveCurrentState()` ต้อง Pop ทั้ง Stack ไปพักไว้ใน Temporary Stack แล้ว Push กลับทุกครั้งที `addAction()` หรือ `getCurrentState()` ถูกเรียก ในขณะที่ Algorithm B ไม่มีการ Pop เกิดขึ้นเลยตลอด การทดลองนี้ เนื่องจากการไม่มีการ Undo เกิดขึ้นในสถานการณ์ทดลอง (มีแต่ `addAction` ต่อเนื่อง) และ `currentState` ถูกอัปเดตแบบ  $O(1)$

สำหรับเวลาเฉลี่ย (Average Time) ผลการทดลองส่วนใหญ่แสดงแนวโน้มที่ Algorithm B เร็วกว่า Algorithm A อย่างชัดเจน (เช่นที่  $n = 10,000$  Algorithm B ใช้เวลาเพียงประมาณ 1 ใน 4 ของ Algorithm A) ยกเว้นที่  $n = 50,000$  ซึ่ง Algorithm A กลับใช้เวลาน้อยกว่า Algorithm B เล็กน้อย ทั้งที่จำนวน Operation ของ A สูงกว่า B มาก ความคลาดเคลื่อนนี้อธิบายได้จาก ปัจจัยของ JVM: การทดลองรัน Algorithm A ก่อน Algorithm B เสมอในแต่ละขนาดข้อมูล ทำให้ Algorithm A ต้องรับภาระ JIT Warm-up (การคอมไพล์ Bytecode เป็น Native Code ระหว่างทำงาน) และ Garbage Collection รอบแรกมากกว่า ซึ่งเป็น Overhead ที่ไม่เกี่ยวกับ Algorithm โดยตรง จึงควรพิจารณาจำนวน Operation

(Push/Pop/Comparisons) เป็นตัวชี้วัดหลักที่แม่นยำกว่า เวลานาฬิกาจริงในการเปรียบเทียบ Algorithm สองแบบนี้ และเป็นข้อจำกัดของการทดลองที่ควรปรับปรุงในบทที่ 8

โดยรวมแล้วผลการทดลองยืนยันว่า Operation Count ของ Algorithm A เติบโตแบบเชิงเส้นตามจำนวนรอบ Workflow ที่ทำ (สอดคล้องกับ  $O(n)$  สะสมต่อรอบ) ในขณะที่ Algorithm B เติบโตแบบเชิงเส้นแต่มีค่าคงที่ (Constant Factor) ต่ำกว่ามาก ซึ่งตรงกับที่คาดการณ์ไว้ในบทที่ 5

## บทที่ 8: สรุป

### 8.1 วิธีที่เหมาะสมกว่าและเหตุผลในการเลือก

กลุ่มเลือก Algorithm B (Event Stack ร่วมกับ State Machine) เป็นแนวทางที่เหมาะสมกว่าสำหรับการใช้งานจริง เนื่องจาก ระบบรับแจ้งเหตุฉุกเฉินต้องอ่านและแสดงสถานะปัจจุบันของเคสบ่อยครั้ง (ทุกครั้งที่เจ้าหน้าที่เปิดหน้าจอหรือระบบตรวจสอบ Action ใหม่) การมี currentState แบบ  $O(1)$  จึงลดภาระการประมวลผลได้อย่างมีนัยสำคัญเมื่อจำนวน Action สะสมมีมาก ผลการ ทดลองในบทที่ 7 ยืนยันว่า Algorithm B ใช้จำนวน Push/Pop/Comparisons น้อยกว่า Algorithm A อย่างสม่ำเสมอในทุกขนาดข้อมูล

อย่างไรก็ตาม Algorithm A ยังมีคุณค่าในบริบทที่ความถูกต้องของข้อมูล (Data Integrity) สำคัญกว่าความเร็ว เนื่องจาก ไม่มี State ซ้ำซ้อนที่อาจไม่ตรงกับข้อมูลจริงหากเกิดข้อผิดพลาดในโค้ดส่วนอื่น จึงเหมาะกับระบบที่มีขนาดเล็กหรือมีการ ตรวจสอบสถานะไม่บ่อยนัก

### 8.2 ข้อจำกัดของงาน

1. Workflow รองรับเฉพาะเส้นทางเชิงเส้นเดียว ไม่รองรับ Workflow ที่มีการแตกสาขาหรือ Action คู่ขนาน
2. ระบบทำงานแบบ In-memory เท่านั้น ไม่มีการบันทึกข้อมูลถาวร (Persistence) หากโปรแกรมปิดตัว ประวัติ Event จะหายไป
3. รองรับเพียง 1 เคสต่อการทำงาน 1 ครั้ง ไม่รองรับหลายเคสพร้อมกัน (Multi-case) หรือ Concurrency
4. การวัดเวลาด้วย System.nanoTime() ยังมี Overhead จาก JVM Warm-up และ Garbage Collection ปะปนอยู่ ทำให้ผลเวลาที่วัดได้มีความคลาดเคลื่อนในบางขนาดข้อมูล ดังที่พบในบทที่ 7

### 8.3 แนวทางพัฒนาต่อ

1. เพิ่มการบันทึก Event Log ลงฐานข้อมูลหรือไฟล์ถาวร เพื่อรองรับการตรวจสอบย้อนหลัง (Audit Trail)
2. ขยายรองรับ Workflow แบบแตกสาขา หรือหลายเคสพร้อมกันด้วยการจัดการ Concurrency ที่เหมาะสม
3. ปรับปรุงการทดลองประสิทธิภาพด้วยการทำ JVM Warm-up ก่อนเริ่มจับเวลาจริง (Discard ผลรอบแรก ๆ) เพื่อลด Noise จาก JIT Compilation

4. พัฒนา REST API หรือ UI แบบกราฟิกแทนเมนู Console เพื่อให้ใช้งานได้สะดวกยิ่งขึ้นในสถานการณ์จริง

8.4 สิ่งที่ได้เรียนรู้

- 1. ความแตกต่างระหว่างการคำนวณสถานะจากข้อมูลต้นทางทุกครั้ง (Derived State) กับการเก็บสถานะเป็น Cache (Cached State) และ Trade-off ด้าน Performance กับ Data Integrity ของทั้งสองแนวทาง
- 2. การใช้ Stack คู่ (Event Stack และ Redo Stack) เพื่อ Implement กลไก Undo/Redo ซึ่งเป็นรูปแบบที่พบได้ทั่วไปในซอฟต์แวร์จริง
- 3. ความสำคัญของ Loop Invariant ในการพิสูจน์ความถูกต้องของอัลกอริทึมที่ทำงานแบบ Incremental ที่ละ Operation แทนที่จะเป็น Loop เดียวที่จบในตัวเอง
- 4. การวิเคราะห์ Big-O ต้องพิจารณาทุกฟังก์ชันของระบบ ไม่ใช่เพียง push()/pop() และผลการทดลองจริงอาจมี Noise จากปัจจัยภายนอก (เช่น JVM Warm-up) ที่ต้องแยกแยะออกจากพฤติกรรมของอัลกอริทึมเอง

ภาคผนวก

ก. ตาราง Test Cases (ผลการทดสอบจริง — รันผ่าน TestCases.java)

ทดสอบทั้งหมด 10 กรณี รันกับทั้ง Algorithm A และ Algorithm B (รวม 20 การทดสอบ) ผลลัพธ์จริง: PASS 20 / FAIL 0

| Test ID | คำอธิบาย                                   | ผลที่คาดหวัง                              | ผลจริง (A) | ผลจริง (B) |
|---------|--------------------------------------------|-------------------------------------------|------------|------------|
| TC01    | Workflow ถูกต้องตั้งแต่เริ่มจนจบ           | สำเร็จทุกขั้นตอน จบที่ CLOSED             | PASS       | PASS       |
| TC02    | เพิ่ม Action ผิดลำดับ (ข้าม TEAM_ASSIGNED) | ถูกปฏิเสธ, currentState ไม่เปลี่ยน        | PASS       | PASS       |
| TC03    | ปิด Case ก่อนถึงที่เกิดเหตุ                | ถูกปฏิเสธ, currentState = DISPATCHED      | PASS       | PASS       |
| TC04    | Undo หนึ่งครั้ง                            | ย้อนกลับหนึ่งขั้น, Redo Stack มี 1 รายการ | PASS       | PASS       |
| TC05    | Undo หลายครั้ง (2 ครั้ง)                   | Redo Stack มี 2 รายการ, state = RECEIVED  | PASS       | PASS       |

|      |                                                |                                              |      |      |
|------|------------------------------------------------|----------------------------------------------|------|------|
| TC06 | Redo บางส่วนหลัง Undo 2 ครั้ง                  | คืน Action ล่าสุดกลับมา,<br>state = ASSIGNED | PASS | PASS |
| TC07 | เพิ่ม Action ใหม่หลัง Redo บางส่วน             | Redo Stack ถูกล้างทั้งหมด                    | PASS | PASS |
| TC08 | Undo จน Event Stack ว่าง แล้ว<br>Undo อีกครั้ง | คืนค่า false, state = NEW                    | PASS | PASS |
| TC09 | Redo เมื่อ Redo Stack ว่าง                     | คืนค่า false                                 | PASS | PASS |
| TC10 | เพิ่ม Action หลังปิด Case แล้ว                 | ถูกปฏิเสธ, state ยังคงเป็น<br>CLOSED         | PASS | PASS |

#### ข. บันทึกการใช้ Generative AI

กลุ่มใช้ Generative AI (Claude, Anthropic) ช่วยในขั้นตอนต่อไปนี้ ตามที่ระบุในตารางด้านล่าง นักศึกษาทุกคนได้ตรวจสอบและทำความเข้าใจโค้ดและเนื้อหาทุกส่วนก่อนนำเสนอ และควรแก้ไขตารางนี้ให้ตรงกับ Prompt และการทำงานจริงของกลุ่มก่อนส่งงาน

| งานที่ใช้ AI ช่วย                                                                    | ผลลัพธ์จาก AI                                                               | วิธีตรวจสอบ                                                                                       | สิ่งที่แก้ไข/เพิ่มเติมโดยนักศึกษา                                                         |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| จัดโครงสร้างและตรวจทาน<br>Java Source Code<br>(AlgorithmA/B,<br>TransitionTable ฯลฯ) | โครงสร้างคลาสตาม Interface<br>เดียวกัน พร้อม Javadoc<br>ภาษาไทยอธิบายแนวคิด | คอมไพล์และรันด้วย<br>OpenJDK 21 จริง, ตรวจสอบ<br>Logic ที่ละเมรอต<br>เทียบกับ Transition<br>Table | (ระบุส่วนที่กลุ่มปรับแก้<br>เพิ่มเติม เช่น การตั้ง<br>ชื่อตัวแปร หรือกฎ<br>เฉพาะของกลุ่ม) |
| สร้าง Test Cases 10 กรณี<br>และรันอัตโนมัติ                                          | TestCases.java ครอบคลุม<br>Normal/Edge/Error Case                           | รันจริงได้ผล PASS 20 /<br>FAIL 0 ตรวจสอบผลลัพธ์<br>แต่ละเคสด้วยมือเทียบกับ<br>Transition Table    | (ระบุ Test Case<br>เพิ่มเติมที่กลุ่มคิดขึ้นเอง<br>หากมี)                                  |
| รันการทดลองประสิทธิภาพ<br>และสร้างกราฟ                                               | ตาราง CSV และกราฟ<br>เปรียบเทียบ                                            | เปรียบเทียบตัวเลขใน<br>กราฟกับค่าดิบใน CSV                                                        | เพิ่มคำอธิบายเรื่อง JIT<br>Warm-up ที่ทำให้ผล                                             |

|                                 | Push/Pop/Comparisons/<br>เวลา                  | และกับการวิเคราะห์<br>Big-O ในบทที่ 5    | เวลาที่ n=50,000<br>คลาดเคลื่อนจากที่คาด                 |
|---------------------------------|------------------------------------------------|------------------------------------------|----------------------------------------------------------|
| ร่างเนื้อหารายงานและสไลด์นำเสนอ | โครงร่างรายงาน 8 บทและสไลด์ 12 หน้าตามข้อกำหนด | อ่านทวนเทียบกับโจทย์และโค้ดจริงทุกหัวข้อ | (ระบุส่วนที่กลุ่มปรับสำนวนหรือเพิ่มเนื้อหาเฉพาะของกลุ่ม) |

การวิเคราะห์ Big-O, การพิสูจน์ Loop Invariant, และข้อสรุปเชิงเปรียบเทียบทั้งหมดในรายงานนี้ผ่านการตรวจสอบและทำความเข้าใจ โดยสมาชิกกลุ่มทุกคน และสมาชิกทุกคนสามารถอธิบาย Source Code ในทุกไฟล์ได้ตามข้อกำหนดของงาน